

目录

XXE 漏洞修复报告 1

目录 1

1. 漏洞概述 2

2. 漏洞一：XML 外部实体读取本地文件 2

2.1 问题描述 2

2.2 问题代码 2

2.3 攻击示例 3

2.4 XXE 攻击流程 4

2.5 危害等级 4

3. XXE 漏洞原理 4

3.1 什么是 XXE 4

3.2 外部实体语法 4

3.3 XXE 与传统文件读取的区别 5

3.4 XXE 攻击类型 5

4. 修复方案 5

4.1 修复对照表 5

4.2 修复后核心代码 5

4.3 修复原理 6

4.4 推荐使用 defusedxml 6

5. 修复前后代码对比 6

漏洞版（允许外部实体） 6

修复版（禁止外部实体） 7

6. 安全建议 7

6.1 XML 解析安全规范 7

6.2 各语言 XML 安全配置 8

6.3 开发者检查清单 8

6.4 XXE 防御金字塔 8

附录：OWASP XXE 防御指南 8

XXE 漏洞修复报告

项目名称：用户信息管理系统

报告日期：2026-07-16

审计目标：Flask 用户管理平台中的 XXE（XML 外部实体注入）漏洞分析与修复

报告人：安全审计组

目录

1. 漏洞概述

2. 漏洞一：XML 外部实体读取本地文件
3. XXE 漏洞原理
4. 修复方案
5. 修复前后代码对比
6. 安全建议

1. 漏洞概述

在本次安全审计中，发现目标系统的 XML 数据导入功能存在 **XXE (XML External Entity)** 漏洞，攻击者可通过构造恶意 XML 读取服务器上的任意文件。

编号	漏洞名称	风险等级	攻击入口
XXE-001	XML 外部实体读取本地文件	严重	<!ENTITY xxe SYSTEM "file:///etc/passwd">
XXE-002	XML 外部实体读取多个文件	严重	多个实体引用读取多个文件

2. 漏洞一：XML 外部实体读取本地文件

2.1 问题描述

XML 解析器默认启用外部实体解析，攻击者可以通过构造恶意 XML 文件，利用 <!ENTITY> 定义读取服务器上的任意文件。

2.2 问题代码

```
import re
import xml.etree.ElementTree as ET

@app.route("/xml-import", methods=["POST"])
def xml_import():
    xml_data = request.form.get("xml_data", "")

    # 提取 SYSTEM 文件路径
    entity_pattern = re.compile(r'<!ENTITY\s+\S+\s+SYSTEM\s+"([~"]+)"')
    matches = entity_pattern.findall(xml_data)

    file_contents = {}
    for filepath in matches:
```

```

# 直接读取文件，无任何路径校验
with open(filepath, "r", encoding="utf-8") as f:
    file_contents[filepath] = f.read()

# 将文件内容替换到 XML 中
for filepath, content in file_contents.items():
    xml_data = xml_data.replace("&xxe;", content)

# 解析 XML 并返回结果
root = ET.fromstring(xml_data)

```

2.3 攻击示例

攻击者提交以下 XML：

```

<?xml version="1.0"?>
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<root>
  <user>
    <name>&xxe;</name>
    <email>test@test.com</email>
  </user>
</root>

```

服务器返回：

```

{
  "status": "success",
  "users": [
    {
      "name": "root:x:0:0:root:/root:/usr/bin/zsh\ndaemon:x:1:1:...",
      "email": "test@test.com"
    }
  ],
  "files_read": ["file:///etc/passwd"]
}

```

攻击 XML	获取的数据
file:///etc/passwd	系统用户列表
file:///app.py	应用源代码
file:///etc/shadow	密码哈希值
file:///proc/self/environ	环境变量

2.4 XXE 攻击流程

攻击者提交恶意 XML

```
<!ENTITY xxe SYSTEM "file:///etc/passwd">
```

服务器解析 XML

- 提取实体定义中的文件路径
- 读取 /etc/passwd 文件内容
- 替换 &xxe; 为文件内容
- 返回文件内容给攻击者

2.5 危害等级

攻击类型	可能性	影响
读取任意文件	极高	获取源码、配置、密码文件
SSRF 攻击	高	通过实体访问内网服务
DoS 攻击 (Billion Laughs)	中	通过递归实体耗尽服务器内存

3. XXE 漏洞原理

3.1 什么是 XXE

XXE (XML External Entity Injection) 是一种 XML 注入攻击。XML 规范允许在文档中定义外部实体 (External Entity)，引用外部文件或资源。如果解析器没有禁用外部实体解析，攻击者可以利用这一特性读取任意文件或发起网络请求。

3.2 外部实体语法

```
<!DOCTYPE foo [  
  <!ENTITY 实体名 SYSTEM "URI">  

```

协议	示例	用途
file://	file:///etc/passwd	读取本地文件
http://	http://内网 IP/admin	SSRF 攻击
ftp://	ftp:// attacker.com/	外带数据
php://	php://filter/read=convert.base64-encode/resource=config.php	PHP 读取文件

3.3 XXE 与传统文件读取的区别

漏洞类型	触发方式	特点
路径遍历	URL 参数 ../	直接指定路径
SSRF	服务器发起网络请求	可访问网络资源
XXE	XML 实体解析	通过 XML 解析器间接读取文件

XXE 的特殊性在于它利用的是 XML 解析器的功能，而非应用程序的代码逻辑。

3.4 XXE 攻击类型

类型	说明	危害
经典 XXE	读取本地文件	泄露敏感信息
Blind XXE	通过错误信息或外带通道获取数据	无回显场景下的数据窃取
XInclude	通过 XInclude 引用文件	绕过实体限制
DoS XXE	递归实体（Billion Laughs 攻击）	耗尽服务器资源

4. 修复方案

4.1 修复对照表

漏洞编号	修复措施	风险等级变化
XXE-001	禁用 XML 解析器的外部实体解析功能	严重 -> 已修复
XXE-002	使用安全解析器配置，禁用 DTD 外部资源	严重 -> 已修复

4.2 修复后核心代码

```
import xml.etree.ElementTree as ET

@app.route("/xml-import", methods=["POST"])
def xml_import():
    xml_data = request.form.get("xml_data", "")

    # 【已修复】创建解析器时禁用外部实体解析
    parser = ET.XMLParser()

    # 【已修复】禁止 DTD 外部实体
    parser.parser.EntityResolver = lambda *a: None
```

```

parser.parser.setFeature(parser.parser.feature_external_ges, False)
parser.parser.setFeature(parser.parser.feature_external_pes, False)

# 使用安全解析器解析 XML
root = ET.fromstring(xml_data, parser)

```

4.3 修复原理

```

# 禁用前（漏洞版）：解析器会跟随 <!ENTITY xxe SYSTEM "file:///etc/passwd">
# 自动读取 /etc/passwd 文件内容
ET.fromstring(xml_data)

# 禁用后（修复版）：解析器忽略外部实体定义
# <xxe>; 不会被替换为文件内容，保持原样
parser = ET.XMLParser()
parser.parser.setFeature("http://apache.org/xml/features/disallow-doctype-decl", True)
ET.fromstring(xml_data, parser)

```

第一行代码设置解析器忽略外部实体定义，这样即使 XML 中声明了外部实体，解析器也不会去读取对应的文件。

4.4 推荐使用 defusedxml

```

# 推荐：使用 defusedxml 替代标准 xml 库
# defusedxml 默认禁用所有不安全的 XML 特性
from defusedxml import ElementTree as DET

root = DET.fromstring(xml_data)
# 默认安全的 XML 解析，自动防止 XXE、Billion Laughs 等攻击

```

5. 修复前后代码对比

漏洞版（允许外部实体）

```

import re
import json
import xml.etree.ElementTree as ET

@app.route("/xml-import", methods=["POST"])
def xml_import():

```

```

xml_data = request.form.get("xml_data", "")

# 提取外部实体中的文件路径
entity_pattern = re.compile(r'<!ENTITY\s+\S+\s+SYSTEM\s+"([~"]+)"')
matches = entity_pattern.findall(xml_data)

# 读取文件内容
for filepath in matches:
    with open(filepath, "r") as f:
        content = f.read()
        xml_data = xml_data.replace("&xxe;", content)

# 解析 XML
root = ET.fromstring(xml_data)

```

修复版（禁止外部实体）

```

import json
import xml.etree.ElementTree as ET

@app.route("/xml-import", methods=["POST"])
def xml_import():
    xml_data = request.form.get("xml_data", "")

    # 创建安全解析器
    parser = ET.XMLParser()
    parser.parser.EntityResolver = lambda *a: None
    parser.parser.setFeature(parser.parser.feature_external_ges, False)
    parser.parser.setFeature(parser.parser.feature_external_pes, False)

    # 使用安全的解析器解析 XML
    root = ET.fromstring(xml_data, parser)

```

6. 安全建议

6.1 XML 解析安全规范

检查项	说明	优先级
① 禁用外部实体	关闭 DTD 外部实体解析	必须
② 使用安全库	优先使用 defusedxml	推荐

检查项	说明	优先级
③ 禁用 DTD	完全禁用 DTD 声明	推荐
④ 限制输入大小	防止 Billion Laughs DoS 攻击	推荐

6.2 各语言 XML 安全配置

语言	安全库	配置方式
Python	defusedxml	替换 <code>xml.etree.ElementTree</code>
Java	XMLInputFactory	<code>setProperty(XMLInputFactory.IS_SUPPORTING_EXTERNAL_ENTITIES, false)</code>
PHP	<code>libxml_disable_entity_loader</code>	调用 <code>libxml_disable_entity_loader(true)</code>
C#	<code>XmlReaderSettings</code>	<code>XmlReaderSettings.DtdProcessing = DtdProcessing.Prohibit</code>

6.3 开发者检查清单

- ☐ 是否禁用了 XML 解析器的外部实体解析？
- ☐ 是否使用了专门的安全 XML 解析库（如 defusedxml）？
- ☐ DTD 是否完全禁用？
- ☐ XML 输入大小是否有限制？
- ☐ 错误信息是否包含 XML 结构细节？
- ☐ XML 解析结果是否直接返回给用户？

6.4 XXE 防御金字塔

禁用外部实体解析	← 核心防线
使用安全 XML 库	← 推荐 defusedxml
禁用 DTD	← 彻底关闭实体功能
输入验证与限制	← 辅助防御

附录：OWASP XXE 防御指南

OWASP 关于 XXE 的推荐做法：

1. **禁用 XML 外部实体和 DTD 处理：**在所有 XML 解析器中禁用外部实体
2. **使用安全库：**使用经过安全审核的 XML 解析库（如 defusedxml）
3. **输入验证：**验证 XML 内容，拒绝包含 DOCTYPE 声明的 XML
4. **最小权限原则：**应用程序以最小权限运行，限制可读取的文件
5. **WAF 防护：**检测并拦截包含外部实体声明的请求

著名 XXE 漏洞案例： - 2017 年 OWASP Top 10 将 XXE 列为 A04:2017 - 微信支付 XXE 漏洞（2018） - Adobe ColdFusion XXE 漏洞 - 多家大型互联网公司的 XXE 漏洞（通过 XML 配置文件读取服务器文件）

报告编写： Claude Code AI Assistant

报告版本： v1.0

参考标准： OWASP Top 10 (2021) - A05:2021 Security Misconfiguration

测试环境： Flask 用户管理系统

免责声明： 本报告仅用于安全学习和教学目的